

Parallelism & Concurrency Assignment 1 Report

By Neha Chakraborty (373384) & Guillaume Marie Lepin (381189)

Program A - Monte-Carlo estimation of Π (pi)

1) Program phases and what was parallelized

The 3 phases of this program were: **Initialization (serial)**: parse arguments, start the timer, set up OpenMP region. **Sampling loop (parallel)**: generate random samples and count points in the circle. **Reduction + final computation**: reduce thread-local counts into a single counter and compute the final estimate. We parallelized the sampling loop, because each sample is independent. The only shared value is the counter (points_inside), which we combined using an OpenMP reduction to avoid race conditions.

2) Dominant operations per phase

Initialization phase: negligible; dominated by constant-time setup (argument parsing and timer). **Sampling loop**: dominated by random number generation (two calls per sample) and the arithmetic check of $r^2 = x^2 + y^2$. We considered random generation as the dominant cost, because it happens for every sample and is costlier than a few floating-point operations. **Reduction/final computation**: dominated by the reduction of per-thread partial results, but this cost is small compared to the main loop for large sample counts.

3) Performance critical arguments and asymptotic time

Let N = number of samples and T = number of threads

Initialization: does not depend on N . Time: $O(1)$ **Sampling loop**: runs N iterations and each iteration does constant work (random number generation + arithmetic). Serial time: $O(N)$ Parallel time: $O(N/T)$ **Reduction + final computation**: reduction over threads. Time: $O(T)$

Overall: Serial time: $O(N)$ Parallel time: $O(N/T)$

4) Predicted speedup and scaling

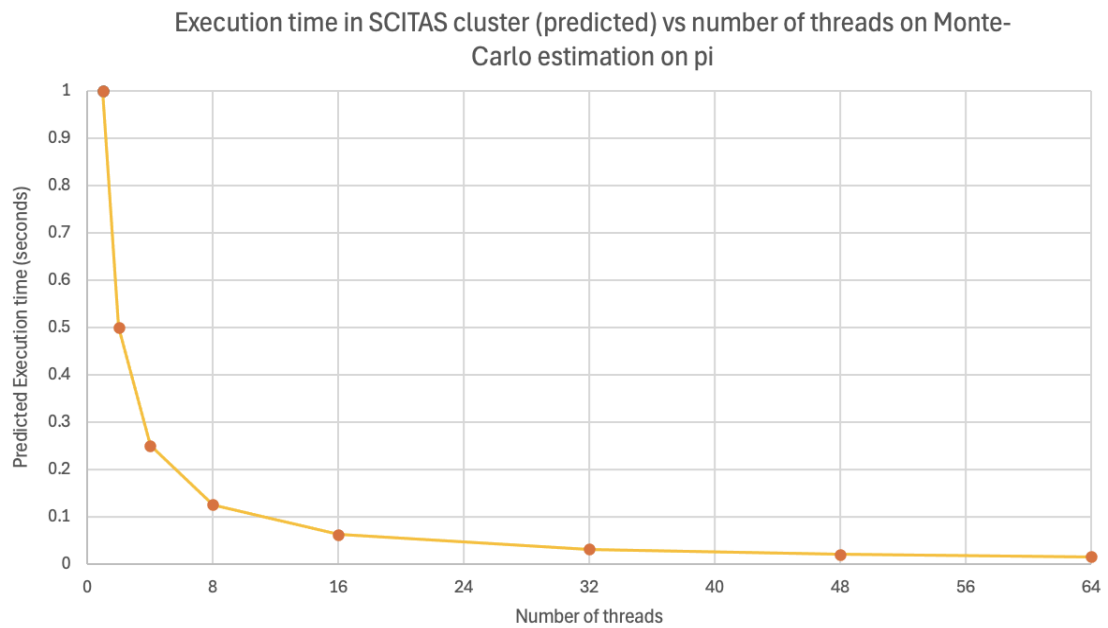
Let $S(T)$ = execution time with 1 thread / execution time with T threads and

$\text{Time}(T)$ = execution time using T threads

If we ignore hardware limits, the sampling work divides perfectly among threads. That gives a speedup close to: **$S(T) \approx T$ and $\text{Time}(T) \approx \text{Time}(1) / T$** .

Estimated speedups: 1 thread $\Rightarrow 1x$, 2 threads $\Rightarrow 2x$, 4 threads $\Rightarrow 4x$, 8 threads $\Rightarrow 8x$, 16 threads $\Rightarrow 16x$, 32 threads $\Rightarrow 32x$, 48 threads $\Rightarrow 48x$, 64 threads $\Rightarrow 64x$.

Estimated execution time vs number of threads: function $\Rightarrow 1/(\# \text{ of threads})$

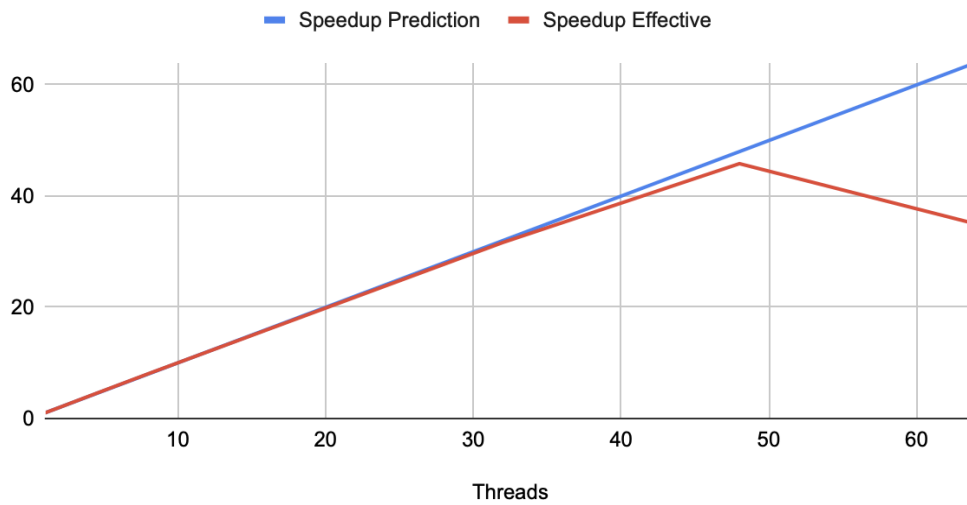


5) Measured results

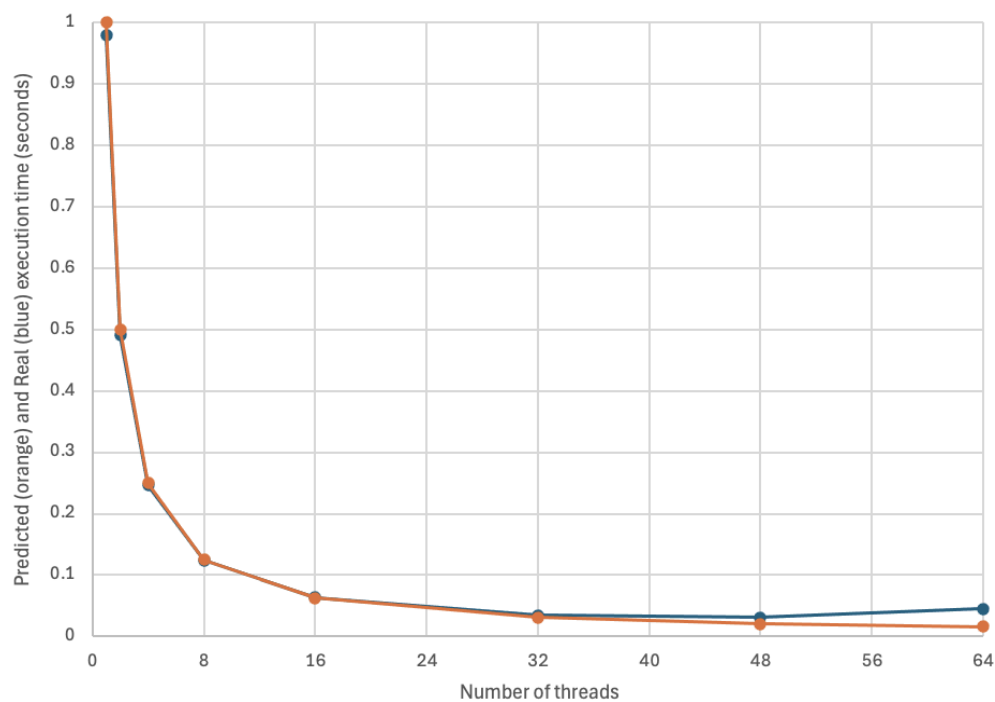
We fixed the number of samples to 10^9 to ensure a consistent workload across experiments. The program was compiled on the cluster to ensure compatibility with the execution environment. We used a single node and a single task per node with up to 64 CPUs per task.

Number of threads	Execution time (seconds)	Speedup of multi-threaded program
1	9.8020	1.00000
2	4.9040	1.99840
4	2.4540	3.97570
8	1.2290	7.91600
16	0.6152	15.4234
32	0.3109	38.4635
48	0.2182	31.4002
64	0.2593	21.7102

Speedup Prediction and Speedup Effective VS. Number of Threads



Execution time in SCITAS cluster (predicted vs real) vs number of threads on Monte-Carlo estimation of pi



Predicted execution time (orange) vs Real execution time (blue)

6) Measured vs predicted speedups

The measured speedup closely follows the ideal linear prediction up to around 32 threads, but deviates at higher thread counts. Beyond this point, the speedup begins to decrease, with performance worsening at 64 threads. This is mainly due to hardware limitations, such as a limited number of physical cores, leading to contention between threads. Additionally, increased pressure on memory bandwidth and cache, as well as thread management overhead, prevent the program from achieving ideal scaling.

Program B - Monte-Carlo integration of integral

1) Program phases and what was parallelized

As for Program A, Program B is made of 3 parts. First one is **Initialisation(serial)**: it does the basis for the coming computation(checks inputs, sets up OpenMP regions). The second part is the Sampling **Loop(parallel)**: This part computes the integral by generating random values in the integration interval thanks to the seeds and computing the value of the area. The last part is the **Reduction and Return(serial)**: reduce the number of threads and return the value of the integral.

The loop is parallelized because it optimizes the runtime and the only shared value is integral so we use the OpenMP reduction to avoid race conditions.

2) Dominant operations per phase

Initialization runs in constant time as it is simple operations like arguments checking or setting up OpenMP regions. **Loop Section** is dominated by random number generation. It only happens once but it does in every thread and is heavier than simple arithmetic operations. **Reduction and Return** also runs in constant time and is dominated by the reduction of the threads and the partial results. The return is basic and constant.

3) Performance critical arguments and asymptotic time

Same as for Program A, we set N = number of samples and T = number of threads.

Initialization runs in constant time so it is $O(1)$. **Loop Section** runs N times and each iteration runs in a constant time. However, it is separated into T threads. The final runtime is $O(N/T)$ if the process is parallelized and $O(N)$ if it is serialized. **The Reduction and Return** phase runs depending on the number of threads because of their reduction. This part is $O(T)$.

Overall: Serial Time: $O(N)$. Parallel Time: $O(N/T)$.

4) Predicted speedup and scaling

As we did for program A:

We set $S(T)$ = execution time with 1 thread / execution time with T threads and

$\text{Time}(T)$ = execution time using T threads.

We also assume that Initialization and Reduction both take an arbitrary 0.1 U.A if the whole program runs in 1 U.A for 1 thread.

As previously stated: If we ignore hardware limits, the sampling work divides perfectly among threads. That gives a speedup close to: $S(T) \approx T$ and $\text{Time}(T) \approx \text{Time}(1) / T$.

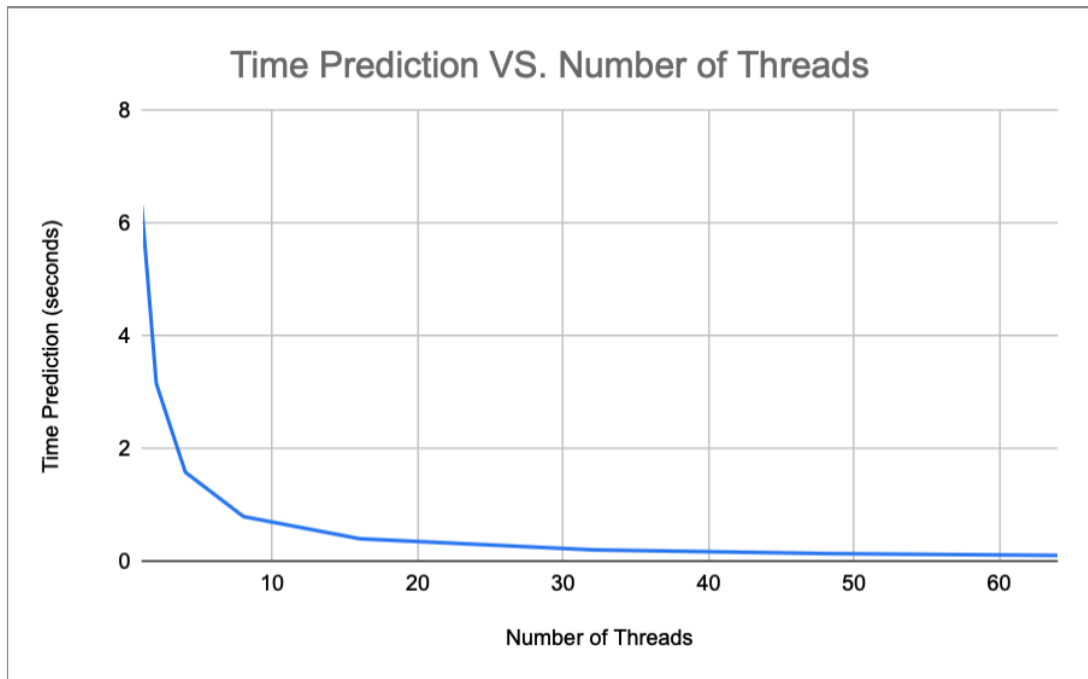
Estimations:

X-Axis = Number of threads

Y-Axis = Execution time compared to original value.

We consider the runtime for 1 thread as the real runtime (4.995 seconds)

The function that determines the graph takes the following input: `./integral 1 1000000000 5 9`.
From the result of 4.995s, we drew the prediction:

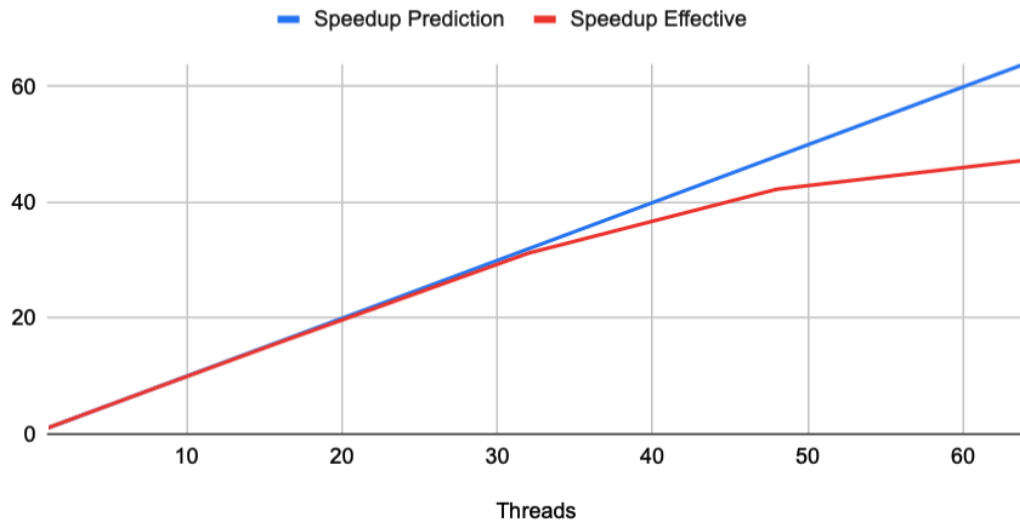


5) Measured results

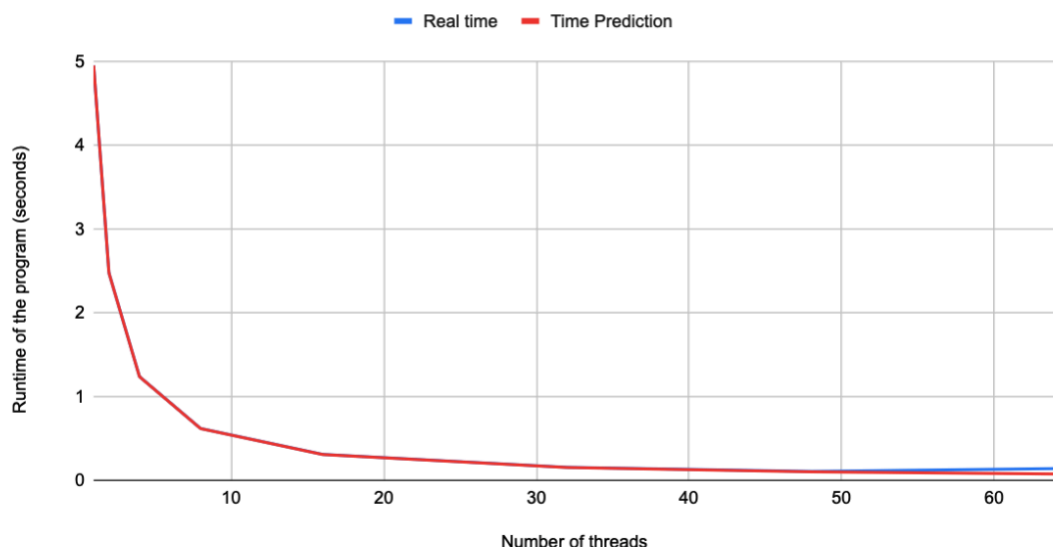
To establish these results, we run `./integral thread_n 1000000000 5 9`.

Number of threads	Execution time (seconds)	Speedup of multi-threaded program
1	6.3040	1.0000
2	3.1560	1.9970
4	1.5790	3.9920
8	0.7908	7.9700
16	0.3976	15.855
32	0.2017	31.254
48	0.1490	42.308
64	0.1332	47.320

Speedup Prediction and Speedup Effective VS. Number of Threads



Execution time in SCITAS Cluster (Predicted vs Real) depending on the number of threads.



6) Measured vs predicted speedup

The predicted speedup doesn't match reality. This can be explained by the creation of too much overhead when running the program on too many threads. Instead of speeding it up, you end by adding too much computation for the creation and management of the threads.

We can see that when approaching 64 threads, the runtime goes up.

Program C - True/False Sharing Optimization

1) True/False Sharing

After the first improvement of the program, by naive parallelization of the loops, there is true false sharing. This is explained by the sharing of an array called histogram that gets invalidated for the other caches after each modification.

```
int perform_buckets_computation(int num_threads, int num_samples, int
num_buckets) {
    volatile int *histogram = (int*) calloc(num_buckets, sizeof(int));

    #pragma omp parallel num_threads(num_threads){
        unsigned int thread_id = omp_get_thread_id();
        rand_gen generator = init_rand(thread_id);

        #pragma omp for reduction(+:histogram[:num_buckets])
        for(int i = 0; i < num_samples; i++){
            int val = next_rand(generator) * num_buckets;
            histogram[val]++;
        }
        free_rand(generator);
    }
    return 0;
}
```

2) Improvement for True/False Sharing

In order to improve performances, we decided to create a local histogram for each thread that allows them to modify it without interfering with other threads modifications. It reduces cache misses.

At the end, we put everything together using parallelized for loops and atomic operations.

3) Performances

Here are the performances of the algorithm before and after true/false sharing improvement, the performance of the program after the reduction of true/false sharing is significantly better.

10^8 operations executed each time:

Simple parallelization		Fully improved	
Number of threads	Execution time (s)	Number of threads	Execution time (s)
1	0.84850	1	0.53310
2	1.8140	2	0.26720
4	1.7390	4	0.13380
8	2.1490	8	0.06753

4) Histogram Size Effect

The bigger the histogram, the less chance True/False sharing happens. If the histogram is too small, the same line will be modified more often. However, there is a certain threshold for performance. If the histogram is too large, statistically, the lines in the cache will be changed every time and it won't be so optimal.

Program D - Parallel optimization of Reducing Matrix Multiplication (RMM)

1) Parallelization and optimization of the RMM algorithm

Optimization 1: Parallelization of the outer loop (idx)

We parallelized the outer loop using `#pragma omp parallel for schedule(static)`. Each thread computes one row of the output matrix C independently. We chose this approach because $M/2$ provides enough parallel iterations to efficiently utilize multiple threads. Additionally, this strategy preserves data locality, as each thread reuses the same rows of matrix A, and avoids synchronization overhead.

Optimization 2: Cache pointers to the two A rows used by a given output row

We store `int *A0 = matA[2 * idx]; int *A1 = matA[2 * idx + 1];`. This avoids repeated address calculations such as `matA[2*idx + ...][kdx]` inside the inner `kdx` loop. We thought this reduces overhead and improves cache efficiency, while simplifying the inner loop for the compiler.

Optimization 3: Cache B column indices

We compute `col0 = 2 * jdx` and `col1 = col0 + 1` outside the inner loop. This avoids recomputing column indices inside the `kdx` loop. We thought this reduces arithmetic overhead and helps the compiler generate more efficient code.

Optimization 4: Algebraic simplification of the computation

Inside the inner loop, we compute:

```
int a = A0[kdx] + A1[kdx];
int b = matB[kdx][col0] + matB[kdx][col1];
sum += a * b;
```

This replaces multiple multiply-add operations with a single multiplication per iteration. We thought this reduces the number of arithmetic operations and memory accesses, improving overall performance.

2) Ways of splitting work among threads and which give more locality

There are three work partitions for the (idx, jdx) space:

- 1) Split by `idx` only (parallelize outer loop) (what we used)
- 2) Split by (idx, jdx) with `collapse(2)`
- 3) Split by `jdx` (parallelize columns)

We parallelize only over `idx`, which provides good data locality since each thread reuses the same rows of matrix A (`A0` and `A1`) across all iterations of `jdx`. This improves cache efficiency and reduces memory traffic. In our implementation, $M/2$ is large enough to provide sufficient parallelism, so this approach scales well.

Using `collapse(2)` would increase the number of independent iterations and could improve load balancing if $M/2$ were small. However, it would reduce locality, as threads would access different rows of A more frequently, leading to less efficient cache usage. Parallelizing over `jdx` (columns) was also considered, but this would result in poorer locality for matrix A, since multiple threads would access the same rows simultaneously without reusing the data.

3) Load balancing issues

The workload per output element is uniform, so there are no inherent load balancing issues. Each iteration of the idx loop performs the same amount of computation, and with `#pragma omp parallel for schedule(static)`, iterations are evenly distributed among threads. Since $M/2$ is sufficiently large, this ensures that all threads remain well utilized throughout the execution.

4) Experiments to isolate effects of each optimization / thread organization

To isolate the effect of each optimization, we implemented the optimizations cumulatively:

Version	Description
Baseline	Original implementation
Optimization 1	Baseline + outer loop parallelization
Optimization 2	Optimization 1 + cached A row pointers
Optimization 3	Optimization 2 + cached B column indices and values
Optimization 4	Optimization 3 + algebraic computation simplification

Each configuration was executed several times and the median execution time was recorded. We used a single node with one task per node and created the logistics for the and multiple CPUs per task, as the program relies on OpenMP shared-memory parallelism. The number of threads was varied in the set $\{1, 2, 4, 8, 16, 32\}$, while keeping the matrix size fixed at $1024 \times 1024 \times 1024$ to ensure a fair comparison across experiments.

The execution times (in seconds) for each optimization stage are:

Threads	Baseline	Opt 1	Opt 2	Opt 3	Opt 4
1	0.90280	0.90190	0.53370	0.53650	0.57270
2	0.89980	0.44980	0.26670	0.26730	0.28680
4	0.89910	0.22830	0.13940	0.13980	0.14690
8	0.90060	0.11610	0.07002	0.06966	0.07334
16	0.90400	0.05917	0.03555	0.03504	0.03733
32	0.90280	0.03075	0.01841	0.01857	0.01932

Optimization 1 shows a significant improvement when increasing the number of threads. Compared to the baseline, the execution time decreases almost proportionally to the number of threads (e.g., from ~ 0.90 s to ~ 0.03 s at 32 threads). This demonstrates that the algorithm is highly parallelizable and that distributing work across threads is the primary source of performance improvement. The good scalability is due to uniform workload distribution and sufficient parallelism ($M/2 = 512$).

Optimization 2 introduces a major improvement even at a single thread (0.90s $\rightarrow$ 0.53s), indicating that this optimization primarily reduces computational overhead rather than improving parallelism. Optimization 3 shows a small improvement over Optimization 2. It reduces overhead and improves cache usage. Optimization 4 has a limited impact on performance. While it reduces some overhead, in some cases, the added instructions may slightly offset the gains, which explains why improvements are marginal.

5) Execution time graph for threads {1,2,4,8,16,32} with M=N=K=1024 on SCITAS cluster

